
TWIN DATACENTER INTERCONNECTION TOPOLOGY

A NEW TWIN-GRAPH-BASED INTERCONNECTION TOPOLOGY FOR SERVER-CENTRIC NETWORKS EXPLOITS GRAPH THEORY TO COST-EFFECTIVELY IMPROVE NETWORK SCALABILITY AND RESILIENCE. A COMPARISON OF TWIN INTERCONNECTION TOPOLOGIES AND OTHER DATACENTER TOPOLOGIES SHOWS THAT THE FORMER HAVE A LOWER LINK COST, SCALE WITH FINE GRANULARITY, AND ARE EFFICIENT, FAULT TOLERANT, AND RESILIENT, BECAUSE THEY ENABLE TRAFFIC BALANCING UNDER BOTH NORMAL AND FAULTY OPERATING CONDITIONS.

.....Modern datacenter applications massively rely on distributed processing and dependable fast migration of large data volumes across a vast domain of physical servers. Therefore, the interconnection topology is a key factor in their design, because it will set the fundamental limits for the datacenter both in terms of cost and performance.

Recently proposed topology models are essentially a hybrid of the two basic interconnection models: Fat Tree,¹ which represents a network-centric design, and Hypercube,² which leverages a server-centric approach. For example, DCell³ and BCube⁴ borrow features from the basic interconnection topology models to reduce number of links and network diameter, respectively. All these basic and hybrid topologies can scale to many elements, but with a coarse granularity because of their regularity patterns in terms of interconnection structures. Moreover, their so-called fault-tolerance features come at a high cost. This fact can be realized, depending on the topology, either in terms of a large set of spare links or excessive increases in hop count whenever faults occur.

We propose an innovative basic topology model called *Twin interconnection* to simultaneously address highly scalable datacenter design with the least possible number of links and high resilience. Twin interconnection can reduce both capital expenditure (CapEx) and operational expenditure (OpEx), and yet provide such networks with graceful performance degradation in the event of element faults.

From the physical network topology viewpoint, fault tolerance is related to the number of node-disjoint paths provided by the network topology for each node pair, whereas resilience also considers the quality of these paths. Thus, two networks with different physical topologies and the same level of fault tolerance can have different levels of resilience. For example, to support any single node or link fault, the network topology must have two node-disjoint paths connecting each node pair. In graph theory terminology, this is called a *2-connected graph*. Generally, these two paths have different lengths, in number of hops. The shortest of them is referred to as the working path, and the other one as the backup path. We refer to

Gilmar L. Vassoler
Federal Institute of
Espírito Santo

Marcia H.M. Paiva
Moisés R.N. Ribeiro
Marcelo E.V. Segatto
Federal University of
Espírito Santo

the difference between their lengths as the hop-count penalty.

A classic 2-connected topology is the ring, for which the hop-count penalty is maximized. Indeed, in a ring with n nodes, for a working path with k hops, $k = 1, \dots, \lfloor \frac{n}{2} \rfloor$, the backup path has $n - k$ hops. For large n , the hop-count penalty can impact network performance and resilience. There is, however, a set of 2-connected topologies with working and backup paths of equal length (that is, a null hop-count penalty) for all non-adjacent node pairs. Within this set, the topologies minimizing the number of links are called *Twin graphs*.⁵ Thus, Twin graphs are optimal 2-connected graphs with respect to the number of links and hop-count penalty.

In this article, we propose the Twin server-centric datacenter network topology, which is based on Twin graphs and which exploits their properties to cost-effectively improve network scalability and resilience. Our results show that Twin topologies have a lower link cost than previous datacenter topologies, requiring less power consumption, and decreasing CapEx and OpEx; are scalable, having the best granularity rate; and are fault tolerant and resilient in flow distribution under both normal and faulty operating conditions.

Twin topologies

Let $G = G(V, E)$ denote a graph consisting of a set $V(G)$ of vertices or nodes, and a set $E(G)$ of edges or links, where each link uv connects a node pair $(u, v) \in G$. The order of G is $n = |V(G)|$, and its size is $m = |E(G)|$. All graphs in this work are connected, undirected, and unweighted. A graph is connected if there is a path connecting every node pair $(u, v) \in G$.

Two nodes $u, v \in V(G)$ are considered adjacent if the link uv belongs to $E(G)$. The neighborhood $\Gamma(v)$ of a node $v \in V(G)$ is the set of nodes adjacent to v in G . The cardinality—that is, the number of elements of $\Gamma(v)$ —defines the degree of v . Two nodes $u, v \in V(G)$ form a twin pair (u, v) in G if and only if they have the same neighborhood.⁵

Every shortest path (in number of hops) connecting two nodes $u, v \in V$ in G is called a $u - v$ geodesic. The diameter $d = d(G)$ is the length of the greatest geodesic in G .

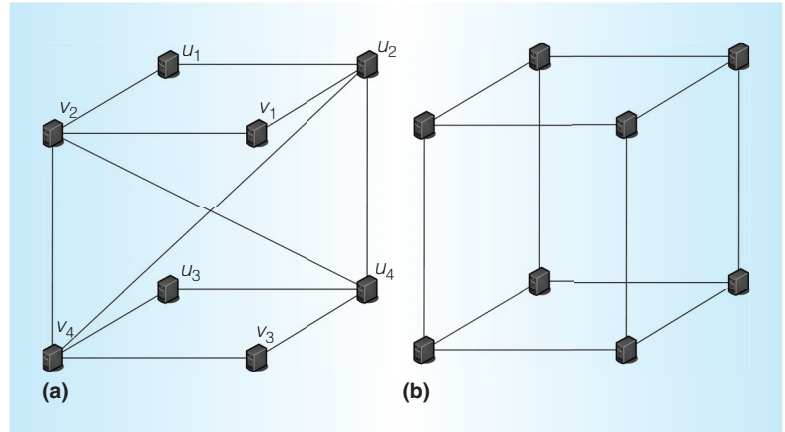


Figure 1. Illustration of server-centric topologies, Twin (a) and Hypercube (b), both with eight nodes and 12 links. The Twin topology has four twin pairs— (u_1, v_1) , (u_2, v_2) , (u_3, v_3) , and (u_4, v_4) —whereas the Hypercube has no twin pairs.

A graph G is 2-connected if and only if there are at least two node-disjoint paths between every two nodes $u, v \in G$. Because node-disjoint paths are also link-disjoint, a 2-connected network topology supports any single node or link fault, since in this case each node pair is still connected by a path avoiding the fault. However, there is no constraint about this path's length, with respect to the length of a geodesic in the original graph. In this way, the class of two geodetically connected graphs has been defined as a special subset of 2-connected graphs.⁶ A graph G is 2-geodetically-connected (2-GC) if and only if there are at least two node-disjoint geodesics connecting every two non-adjacent nodes $u, v \in G$. Thus, in the case of a single node or link fault, an alternative geodesic still connects every pair of nonadjacent nodes, avoiding the faulty element.

Now, Twin graphs can be defined as follows: every Twin graph is a minimum-size 2-GC graph. Each one has order $n \geq 4$ and size $m = 2n - 4$.⁵ Twin graphs benefit from building processes based on the concept of twin pairs. For instance, by means of a twin pair, a node can always be added to a Twin graph to form a new one, and it generally can be done in several ways, because twin pairs are not unique. Indeed, for each n , there is a family of Twin graphs.⁵ Moreover, two Twin graphs can always merge to form a third one that requires only four additional links.

Figure 1a shows a Twin graph of order 8, where it can be verified that it has 12 links,

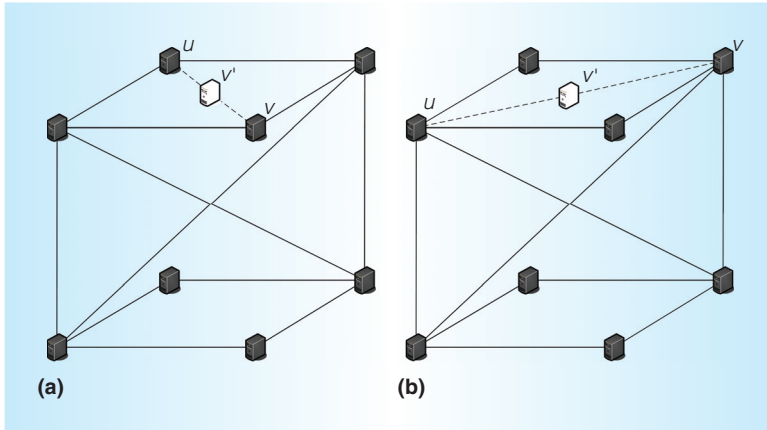


Figure 2. Growing a Twin topology. Dotted lines represent the two possible ways (up to isomorphism) of growing the Twin topology of order 8 shown in Figure 1a to build a Twin topology of order 9.

four twin pairs, and at least two node-disjoint geodesics connecting any pair of nonadjacent nodes. The Hypercube shown in Figure 1b also has eight nodes, 12 links, and at least two node-disjoint geodesics for each nonadjacent node pair; however, it has no twin pairs. Thus, it cannot benefit from the Twin building processes.

Twin growing process

We can recursively define Twin graphs as follows:⁵ the cycle of order 4 is a Twin graph; if G is a Twin graph of order n , and (u, v) is a twin pair in G , then the addition of a new node v' by using links uv' and vv' produces a Twin graph G' of order $n + 1$. Moreover, (u, v) remains a twin pair in G' . Thus, the process to grow a Twin graph G is as follows:

- Identify a twin pair (u, v) in G .
- Build G' , where $V(G') = V(G) \cup \{v'\}$ and $E(G') = E(G) \cup \{uv', vv'\}$.

Figure 2 shows the two possible ways (up to isomorphism) for growing the Twin graph of order 8 shown in Figure 1a to build a Twin graph of order 9.

Twin merging process

The process to merge two Twin graphs G_1 and G_2 is as follows:

- Identify a twin pair (u_1, v_1) in G_1 and a twin pair (u_2, v_2) in G_2 .
- Build $G = G(V, E)$, where $V = V(G_1) \cup V(G_2)$ and $E = E(G_1) \cup E(G_2) \cup \{u_1u_2, u_1v_2, u_2v_1, v_1v_2\}$.

Figures 3a and 3b illustrate this process in two ways. The white servers represent the selected twin pairs, and the dotted lines represent the four new links. If the main concern is the diameter, the designer can merge two Twin graphs from Figure 1a, as shown in Figure 3a. On the other hand, if the main concern is the maximum node degree, the designer can choose Figure 3b. Naturally, the resulting Twin graphs have 16 nodes and 28 links. Figure 3c represents the default merging process of two Hypercubes from Figure 1b, which cannot use the Twin merging process, because Hypercubes have no twin pairs. The graph resulting from this process also has 16 nodes, but 32 links. It shows the advantage of Twin graphs over Hypercubes with respect to cost. It is also important to emphasize that Twin graphs allow several ways of merging, which makes the interconnection topology design more flexible.

Datacenter network topology design

We define the following aspects of the datacenter network topology design problem: cost, scalability, fault tolerance, resilience, and performance.

Cost

The datacenter network cost varies according to the hardware equipment used to forward or process network traffic. NIC (Network Interface Card) and cabling costs are present in all network-centric, server-centric, and hybrid architectures. However, the server-centric architecture eliminates networking equipment, such as switches, routers, and load balancers, thanks to this architecture's capability to multiplex CPU cores across applications and network-related processing.⁷ This reduction can reach 15 percent of the datacenter total cost, and makes the server-centric architecture one of the most cost-attractive interconnects.⁸ A drawback of server-centric architectures is the burden of multihop forwarding on server computation resources. However, we can address this by replacing some copper links with 2×2 optical switches.⁹ Therefore, we investigate a Twin server-centric datacenter network topology in this article, but Twin graphs could also be applied to network-centric or hybrid datacenter interconnects.

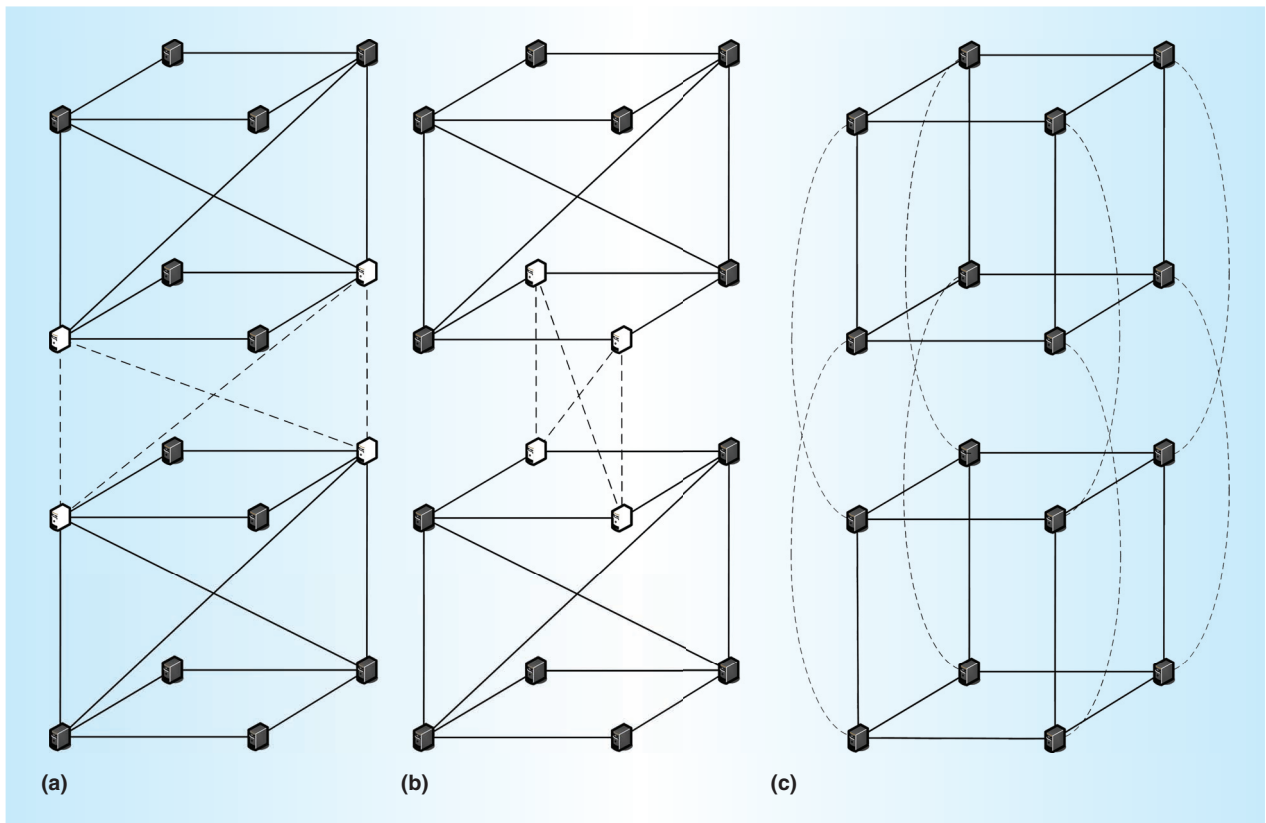


Figure 3. Merging process for Twin graphs and Hypercube: Twin merging process with high-degree node and low diameter (a); Twin merging process with low-degree node and high diameter (b); two Hypercubes merged together (c). In both (a) and (b), white servers represent twin pairs, and dotted lines represent the new links to be added. Thus, each resulting Twin topology has 16 nodes and 28 links. In (c), dotted lines represent the eight links to be added to merge two Hypercubes of Figure 1b. The resulting Hypercube also has 16 nodes, but 32 links.

In this work, to compare costs for different architectures, equal commodity servers are considered for all topologies, and only the number of links needed to interconnect all servers is computed. The number of links relates directly to CapEx (for example, cabling and port count), and indirectly to OpEx (for example, overall power consumption of ports and maintenance of fewer links and ports).

Scalability

During a datacenter's lifetime, incremental expansion of the network can be necessary to accommodate, for example, demands of new users, or can be motivated by new bandwidth-intensive applications. In general, companies incrementally expand commodity datacenters by adding more servers to the

network. However, commodity datacenters such as Fat Tree, DCell, BCube, and Hypercube do not support the addition of an arbitrary number of servers. All these topologies can achieve a huge number of servers, but their incremental expansion requires the addition of as many servers as needed to fill the topology, even when only a few servers would suffice.

The incremental expansion problem is overcome using Twin topologies, because of their growing and merging processes. Thus, when the datacenter must be expanded slowly, we can add precisely the number of servers required, using the Twin growing process; when a fast expansion is needed, we can use a modular datacenter approach (such as arranging the servers in containers), using the Twin merging process to connect the modules.

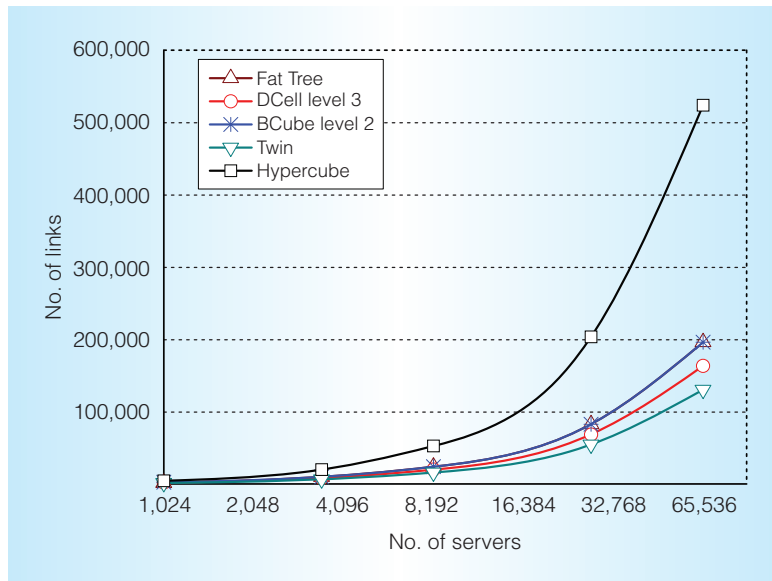


Figure 4. Number of links versus number of servers for Fat Tree, BCube, DCell, Hypercube, and Twin topologies. A large datacenter can benefit from fewer links required by Twin topologies.

Fault tolerance, resilience, and performance

To provide uninterrupted services with the best performance, datacenter networks must be fault tolerant and resilient—that is, they not only need to support faults while remaining connected, but they also need to maintain performance after a fault.

Fault tolerance is related to the number of node-disjoint paths the network topology provides for each node pair, whereas resilience also accounts for the quality of these paths. DCell, BCube, Hypercube, and Twin topologies provide at least two node-disjoint paths for each node pair. On the other hand, Fat Tree has a lower level of fault tolerance, because each host connects to the network via a single link through edge switches. These topologies' performance is affected differently after a single fault.

In this work, we measure the network performance by the flow completion time¹⁰ to end all connections, obtained from emulation. We measure the resilience by the increase in flow completion time under faulty operating conditions, and also by hop-count penalty and network diameter increase.

Results and discussion

We compare Twin topologies to Fat Tree, DCell, BCube, and Hypercube topologies,

with respect to the aspects we discussed earlier.

Cost

To compare costs for Fat Tree, BCube, DCell, Hypercube, and Twin topologies with n servers, we computed the number of links needed to interconnect all servers. Fat Tree topologies using commodity switches can typically be built for $n = 1,024$; 3,456; 8,192; 27,648; or 65,536 servers. So, we adopted these values for all topologies.

The number of links for Twin topologies is $2n - 4$. For the other topologies under study, we compute this using their parameters (switch port number for Fat Tree, level and switch port number for DCell and BCube, and node degree for Hypercube).

Figure 4 shows the number of links with respect to the number of servers for all these topologies. Twin topologies use the minimum number of links, followed by DCell. Fat Tree and BCube, for which the curves are overlapping, have almost the same number of links; and Hypercube presents the largest number of links. For 65,536 servers, the number of links in Twin topologies is 20 percent lower compared to DCell and 75 percent lower compared to Hypercube.

Scalability

We can analyze the scalability of commodity datacenters from their typical number of servers n (see Table 1). For Fat Tree, n depends on the number of ports of commodity switches; for DCell and BCube, n depends on the level and number of ports of commodity switches; and for Hypercube, it is related to the node degree δ . For instance, a datacenter with 10,000 servers using Fat Tree built with 48-port switches will provide capacity for interconnecting 27,648 servers. For DCell and BCube topologies, we can find a combination between the number of switch ports and the level to achieve a port count closer to 10,000. For Hypercube, the node degree must be 14, which needs 16,384 servers in the network.

Another drawback of Fat Tree, DCell, and BCube is when new servers must be added in a fully used network topology. For example, if 1,000 extra servers must be accommodated in Fat Tree 16-port switches, all switches

Table 1. Typical number of servers for Fat Tree, DCell, BCube, and Hypercube topologies, using commodity switches.

Fat Tree	No. of servers	DCell	No. of servers	BCube	No. of servers	Hypercube	No. of servers
SW port 16	1,024	Level 2, SW port 6	1,806	Level 2, SW port 16	4,096	$\delta = 12$	4,096
SW port 24	3,456	Level 2, SW port 8	5,256	Level 2, SW port 24	13,824	$\delta = 13$	8,192
SW port 32	8,192	Level 2, SW port 10	12,210	Level 2, SW port 32	32,768	$\delta = 14$	16,384
SW port 48	27,648	Level 2, SW port 12	24,492	Level 3, SW port 16	65,536	$\delta = 15$	32,768
SW port 64	65,536	Level 3, SW port 4	176,820	Level 3, SW port 24	331,776	$\delta = 16$	65,536

* δ represents the node degree.

must be replaced by 24-port switches, and then 2,432 new ports will be available. The DCell and BCube have a similar problem, whereas for Hypercube interconnects the number of servers can only be doubled. Although it is possible to build a topology with any partial implementation, there is no guarantee that a pruned topology will present the same resilience and performance of its complete implementation.

In addition, using Twin topologies, there are no overprovisioned ports, the datacenter can have an arbitrary number of servers, and new servers can be added in any number.

Fault tolerance, resilience, and performance

To compare the performance of Twin, Fat Tree, DCell, BCube, and Hypercube topologies, we built a model for each topology in Mininet.¹¹ We implemented an Openflow controller, which allowed the performance evaluation of these topologies using two different routing approaches: OSPF (Open Shortest Path First) and ECMP (Equal Cost Multipath Protocol). We used the same routing approaches for all topologies to make the result as fair as possible, because our main concern is the physical topology. Finally, we adopted a topology-unbiased uniform traffic scenario (that is, all-to-all communication).

We tested the models using 16 servers, and the traffic for each node pair would represent a 500-Mbyte flow, sent at the maximum interface rating (that is, 1 Gbps). For comparison purposes, we also tested a full mesh model using these parameters. In the case of DCell, 20 servers and 316 Mbytes per flow were used to achieve the same amount of exchanged

traffic (that is, 120 Gbytes). We generated the individual flows using Iperf (<http://iperf.fr>). For this scenario, Fat Tree, DCell, BCube, Hypercube, and Twin topologies have 36, 30, 32, 33, and 28 links, respectively.

For 16 servers, the Twin family has 182 members.¹² We tested them all, and chose two of them (Twin-1 and Twin-2) to show the results. Twin-1 minimizes the diameter (but maximizes the maximum node degree), whereas Twin-2 has the same diameter as BCube and Hypercube.

First, we tested the performance of each topology under normal operation. The first and fourth bars in Figure 5 show the flow completion time using OSPF and ECMP, respectively. The full mesh topology spent 70 seconds to end all 240 flows, as indicated by the dotted line in Figure 5. The best performance for OSPF was 73 seconds, whereas for ECMP it was 72 seconds. For Twin topologies, the time spent varies with the chosen member of the family. Twin-1 spent 73 seconds in both approaches. It is very close to the time spent by full mesh, and cannot be improved by ECMP, because of the forwarding of flows between nonadjacent servers. EMCP performs well for Twin-2, Fat Tree, and Hypercube topologies, because these topologies have at least two geodesics for most node pairs. However, BCube and DCell have only one geodesic for each node pair. That is why Figure 5 shows little or no improvement for such interconnection topologies under ECMP. It is important to note that, even for this small setup, Twin topologies reduce up to 28 percent the number of links, with performance under ECMP very close to the other topologies.

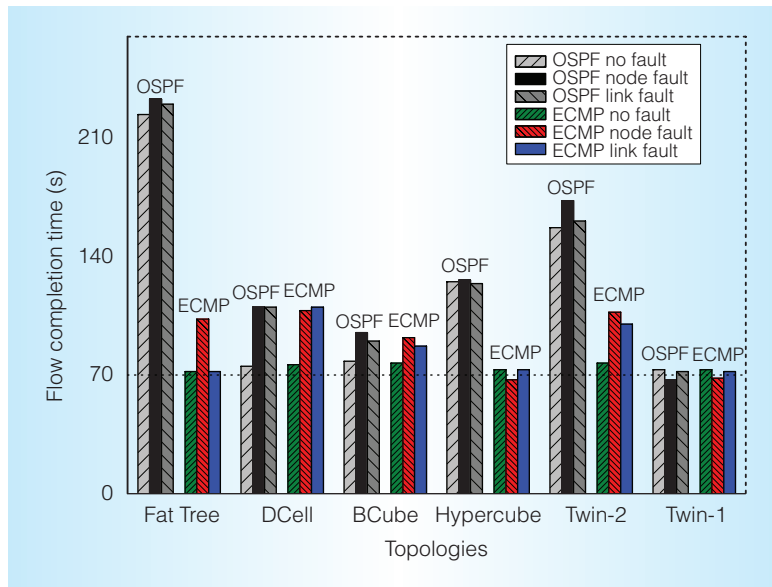


Figure 5. Flow completion time to end all connections in each topology, under normal and faulty operating conditions, using Open Shortest Path First (OSPF) and ECMP (Equal Cost Multipath Protocol) approaches. The dotted line represents the flow completion time for a full mesh topology to end all 240 flows.

To analyze the same scenario under faulty operation, we introduced either a maximum-degree node fault or a random link fault.

All 2-connected topologies under study have the same level of fault tolerance (that is, they support at least one single node or link fault that remains connected). However, they present different levels of resilience. We measure resilience by the increase in flow completion time under faulty operating conditions, and by hop-count penalty and network diameter increase.

In Figure 5, the second and fifth bars show the flow completion time for each topology after a node fault, using OSPF and ECMP, respectively; the third and sixth bars show the corresponding results after a link fault. When comparing these topologies' flow completion times, we see that Twin-1 had the overall best performance for both link faults and node faults, using OSPF. For ECMP, Twin-1, Fat Tree, and Hypercube had practically the same performance for link faults. However, for node faults, Twin-1 and Hypercube performed better than Fat Tree. Finally, in topologies without switches, such as Twin and Hypercube, a node fault reduces

the total traffic, which could reduce the flow completion time.

Figure 6 shows the flow distribution with respect to the number of hops, from which we analyze hop-count penalty and network diameter. The triples represent (d, d^n, d^l) , where d is the network diameter, d^n is the diameter after a maximum-degree node fault, and d^l is the diameter after a random link fault. For the topologies with at least two geodesics for most node pairs, the diameter does not change substantially after a fault. For BCube and DCell, however, the diameter increases in two units.

The flow distribution with respect to number of hops is almost the same in Twin topologies, even in fault situations, whereas for BCube and DCell, the number of flows using longer paths is increased (and, consequently, the average network delay is higher). The connections using six or seven hops in Figure 6b, and the connections using six hops in Figure 6c, illustrate the effect of the hop-count penalty on resilience. Fat Tree also has graceful performance degradation, but its flow distribution shows most connections using long paths. Finally, in terms of flow distribution, diameter, and hop-count penalty, the results for Hypercube and Twin-2 topologies are similar.

In addition, note that a node fault never leads to diameter increase in Twin topologies, by definition. Even in the worst case, where the server with the greater number of connections fails, all other servers can communicate using shortest paths. These results ensure fault tolerance, resilience, and performance of Twin datacenter network topologies.

Practical implications and future research directions

We discuss how to choose and identify a Twin topology, build large Twin topologies, foresee cabling, and handle large-degree nodes in a Twin topology.

How to choose a Twin topology

When designing a Twin datacenter with n servers, there is a family of Twin topologies of order n to choose from, according to the design's particular requirements. To make it easier to choose the topology, we can reduce

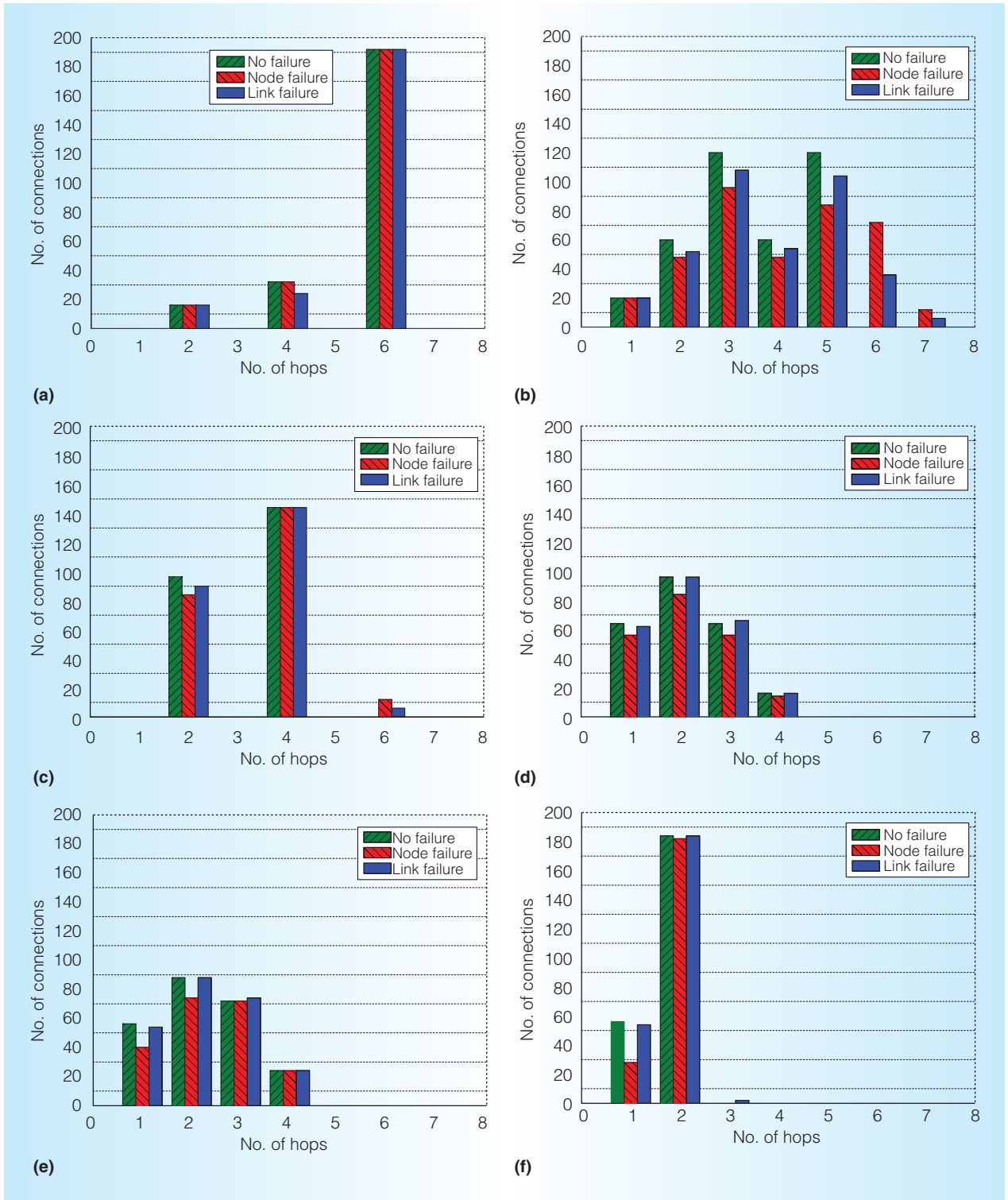


Figure 6. Flow distribution with respect to number of hops. The triples represent (d, d^n, d^l) , where d is the network diameter, d^n is the diameter after a maximum-degree node fault, and d^l is the diameter after a random link fault. The diameter triples for each topology are as follows: Fat Tree, (6, 6, 6) (a); DCell, (5, 7, 7) (b); BCube, (4, 6, 6) (c); Hypercube, (4, 4, 4) (d); Twin-2, (4, 4, 4) (e); and Twin-1, (2, 2, 3) (f).

this family by using one or a sequence of filters, such as maximum node degree, diameter, and average number of hops. Moreover, in a Twin family of order n , it is possible to find a good tradeoff between diameter and maximum node degree, as shown in Figures 3a and 3b for a small network. For each n , there is a Twin topology of diameter 2,⁵ and there is at least one Twin topology with a diameter ranging from 2 to $\lfloor \frac{n}{2} \rfloor$ for each $n \leq 17$.¹²

How to identify a Twin topology

Twin topologies are scalable because of the growing and merging processes, but both processes must start with Twin topologies. Fortunately, it is computationally easy to identify Twin topologies. Using the recognition algorithm provided by Chang and Ho,¹³ we can decide if a topology of order n and size $m = 2n - 4$ is a Twin topology in $O(mn) = O(n^2)$.

We could apply this algorithm to rewire a given topology with n nodes into a Twin graph by simply rearranging its interconnections, or upgrading or downgrading its number of links.

How to build large Twin topologies

It is easy to build Twin topologies of order n , node by node, from the cycle of order 4, or by merging two Twin topologies of orders n_1 and n_2 , where $n = n_1 + n_2$. Thus, given n , we could build all Twin topologies of order n , and then choose the one that best fits the requirements of the design. However, the node-by-node recursive process is not recommended for large n , say, for hundreds or thousands of nodes, because the number of Twin topologies increases rapidly with n . Furthermore, using the merging process, there is no guarantee of getting the best Twin topology. Therefore, the practical application of large Twin datacenters depends on the development of efficient algorithms to build large Twin topologies.

How to foresee cabling arrangement complexity in a Twin topology

Twin graphs provide the minimal link-counting interconnection topology, which means a substantial reduction of the cabling complexity. Minimally invasive processes are needed for growing and merging Twin

topologies, as illustrated in Figures 2 and 3, respectively. Moreover, because all Twin graphs are planar,¹⁴ there is always a spatial node arrangement such that no link crosses another. All these features could favor arranging the cables more compactly, and even layered arrangements can be investigated thanks to Twin's planarity. In other words, cables can be laid in such a way that no network interconnections cross each other.

How to handle large-degree nodes in a Twin topology

Servers usually have a limited number of slots for NICs, so it is difficult for them to accommodate high-degree nodes in server-centric architectures. Hybrid interconnection models such as DCell and BCube use switches to extend the server's degree, as shown in Table 1. We could propose the same strategy to provide high-degree nodes in hybrid architectures based on Twin topologies. In contrast with regularly connected DCell and BCube, there are Twin topologies for which only few nodes have the maximum degree, as shown in Figure 3a.

Twin topologies offer a viable alternative for server-centric, network-centric, or hybrid datacenter interconnects. Future work will address efficient algorithms to build large Twin topologies, as well as techniques for rearranging a given topology into Twin topology. Twin graph planarity will be also investigated for modular datacenters aiming at given performance metrics that are relevant to datacenters.

MICRO

Acknowledgments

We thank FCT/CAPES (Grant 376/14) and FAPES/CNPq (Grant 48508569) for their financial support. We thank the Federal Institute of Espírito Santo (IFES) for allowing Gilmar Vassoler to work on his PhD thesis. Marcia Paiva was partially supported by CAPES Grant PNP20130772. Marcelo Segatto was partially supported by CNPq Grant 307470/2012-1.

References

1. M. Al-Fares, A. Loukissas, and A. Vahdat, "A Scalable, Commodity Data Center

- Network Architecture," *Proc. ACM SIGCOMM Conf. Data Communication*, 2008, pp. 63-74.
2. M.Y. Saad and M.H. Schultz, *Data Communication in Hypercubes*, tech. report, DTIC, 1985.
 3. C. Guo et al., "DCell: A Scalable and Fault-Tolerant Network Structure for Data Centers," *Proc. ACM SIGCOMM Conf. Data Communication*, 2008, pp. 75-86.
 4. C. Guo et al., "BCube: A High Performance, Server-Centric Network Architecture for Modular Data Centers," *Proc. ACM SIGCOMM Conf. Data Communication*, 2009, pp. 63-74.
 5. A. Farley and A. Proskurowski, "Minimum Self-Repairing Graphs," *Graphs and Combinations*, vol. 13, no. 4, 1997, pp. 345-352.
 6. R. Entringer, D. Jackson, and P. Slater, "Geodetic Connectivity of Graphs," *IEEE Trans. Circuits and Systems*, Aug. 1977, pp. 460-463.
 7. L. Popa et al., "A Cost Comparison of Data-center Network Architectures," *Proc. 6th Int'l Conference (Co-NEXT 10)*, 2010, article 16.
 8. A. Greenberg et al., "The Cost of a Cloud: Research Problems in Data Center Networks," *SIGCOMM Computer Comm. Rev.*, vol. 39, no. 1, 2009, pp. 68-73.
 9. G.L. Vassoler et al., "Hybrid Reconfiguration for Upgrading Datacenter Interconnection Topology," *Proc. IEEE Photonics Conf.*, 2012, pp. 782-783.
 10. N. Dukkkipati and N. McKeown, "Why Flow-Completion Time is the Right Metric," *SIGCOMM Computer Comm. Rev.*, vol. 36, no. 1, 2006, pp. 59-62.
 11. B. Lantz, B. Heller, and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," *Proc. 9th ACM SIGCOMM Workshop Hot Topics in Networks*, 2010, article 19.
 12. M.H.M. Paiva, G. Caporossi, and M.E.V. Segatto, "Twin Graphs for OTN Physical Topology Design," *Les Cahiers du GERAD* [Reports of the Group for Research in Decision Analysis], vol. 48, 2013, no. G-2013-48.
 13. J.-M. Chang and C.-W. Ho, "The Recognition of Geodetically Connected Graphs," *Information Processing Letters*, vol. 65, no. 2, 1998, pp. 81-88.
 14. J. Plesník, "Towards Minimum K-Geodetically Connected Graphs," *Networks*, vol. 41, no. 2, 2003, pp. 73-82.
- Gilmar L. Vassoler** is a lecturer in the Department of Informatics at the Federal Institute of Espírito Santo and a doctoral candidate in the Telecommunications Laboratory (LabTel) at the Federal University of Espírito Santo. His research interests include datacenter networks architecture, optical networks for datacenters, cloud computing, and virtualization. Vassoler has an MSc in electrical engineering from the Federal University of Espírito Santo.
- Marcia H.M. Paiva** is a postdoctoral researcher in the Telecommunications Laboratory at the Federal University of Espírito Santo. Her research interests include network topology, graph theory, and optical networks. Paiva has a PhD in electrical engineering from the Federal University of Espírito Santo.
- Moisés R.N. Ribeiro** is a professor in the Department of Electrical Engineering at the Federal University of Espírito Santo. His research interests include fiber-optic communication and sensor devices, systems, and networks. Ribeiro has a PhD in electrical engineering from the University of Essex.
- Marcelo E.V. Segatto** is a professor in the Department of Electrical Engineering at the Federal University of Espírito Santo. His research interests include fiber-optic communication devices, systems, and networks. Segatto has a PhD in physics from Imperial College London. He is a member of the Optical Society of America.
- Direct questions and comments about this article to Gilmar L. Vassoler, Universidade Federal do Espírito Santo, Centro Tecnológico, Departamento de Engenharia Elétrica, Av. Fernando Ferrari, 514—Laboratório de Telecomunicações, Goiabeiras, 29075-910, Vitória, ES, Brazil; gilmar-vassoler@ifes.edu.br.